instituto superior de
engenharia de lisboa

Departamento de Engenharia de Electrónica e Telecomunicações e de Computadores

Segundo Trabalho Prático

51445 : José Saldanha (a51445@alunos.isel.pt)

51447 : Ricardo Pinto (a51447@alunos.isel.pt)

51484 : Vasco Pereira (a51484@alunos.isel.pt)

Relatório para a Unidade Curricular de Segurança Informática
da Licenciatura em Engenharia Informática e de Computadores

Professor : Dr. José Simão

12 – dezembro – 2025

Contents

1 Exercício 1 **1**

1.1 Quebra de Passwords com John the Ripper 1

1.2 Execução com opções por omissão 1

1.3 Ataque com dicionário 2

1.4 Ataque direcionado à password da mallory 2

2 Exercício 2 **5**

2.1 Estado de autenticação no Moodle 5

2.2 Acesso ao perfil usando outro cliente 6

3 Exercício 3 **7**

3.1 Integração OAuth2 e OpenID Connect do Calendly 7

3.2 Autenticação com Microsoft 7

3.3 Autenticação com Google 8

3.4 Comparação entre Microsoft e Google 9

3.5 Presença do access token e id token no browser 9

4 Exercício 4 **11**

4.1 Política de controlo de acessos 11

4.2 Análise de acesso 11

4.3 Conclusão 12

5 Exercício 5 **13**

5.1 Configuração de um servidor HTTPS e autenticação de cliente 13

5.1.1 Configuração sem autenticação de cliente 13

5.1.2 Configuração com autenticação de cliente 14

5.1.3 (i) Algoritmos de segurança usados no Record Protocol 15
5.1.4 (ii) Vulnerabilidade da cipher suite TLS_RSA_WITH_AES_128_CBC_SHA256 . . . 16
5.2 Ligação a servidor HTTPS usando JCA 16

6 Exercício 6 **19**
6.1 Aplicação Web GitTasks 19
6.2 Configuração e variáveis de ambiente 19
6.3 Autenticação de utilizadores 20
6.4 Autorização e papéis (RBAC) 20
6.4.1 Atribuição de papéis 20
6.4.2 Permissões 21
6.5 Integração com GitHub 21
6.6 Integração com Google Tasks 21
6.7 Sessões e armazenamento em memória 21
6.8 Interface Web 22
6.9 Segurança e controlo de acesso 22
6.10 Resumo 22

1

Exercício 1

1.1 Quebra de Passwords com John the Ripper

Neste exercício analisam-se os resultados da quebra de passwords presentes nas bases de utilizadores `md5.txt` e `bcrypt.txt`, ambas armazenando hashes com *salt*. Os testes foram realizados com a ferramenta `John the Ripper`, utilizando diferentes modos de ataque.

1.2 Execução com opções por omissão

O comando executado para cada base foi:

```
$ john NOME_FICHEIRO
```

As execuções decorreram durante pelo menos 10 minutos, conforme solicitado.

Resultados sobre bcrypt

Foram quebradas três passwords:

- eva – 123456
- bob – 123456789
- carol – password

Todas as passwords foram quebradas em milissegundos entre si, dado o custo reduzido (5) e a fraqueza das passwords. A password mais lenta a ser quebrada foi a da utilizadora **eva**.

Resultados sobre MD5

Foram igualmente quebradas as mesmas três passwords:

- eva – 123456
- bob – 123456789
- carol – password

Também neste caso a password da **eva** foi a mais lenta.

Não se observaram diferenças significativas de desempenho entre MD5 e bcrypt (com custo baixo), pois todas as passwords eram fracas.

1.3 Ataque com dicionário

O comando utilizado foi:

```
$ john --wordlist=rockyou.txt --rules NOME_FICHEIRO
```

A wordlist `rockyou.txt` e as regras associadas permitiram quebrar passwords adicionais:

- trudy – (quebrada através do dicionário)
- alice – (quebrada através do dicionário)

A password da utilizadora **mallory** permaneceu não quebrada após este ataque.

1.4 Ataque direcionado à password da mallory

A password da mallory foi gerada aleatoriamente pelo site `random.org`, que apresenta ao utilizador 5 possíveis passwords. Supondo que um atacante obteve essa lista, foi usada a seguinte wordlist:

```
possiblePasswords.lst
```

O comando executado foi:

```
$ john --wordlist=possiblePasswords.lst NOME_FICHEIRO
```

Os resultados foram os seguintes:

- A password da **mallory** foi quebrada com sucesso.
- O tempo necessário foi praticamente nulo, dado que apenas existiam cinco possíveis passwords.
- Nenhuma outra password foi quebrada, dado que as restantes não constavam na wordlist.

Conclusões

- As passwords fracas foram facilmente quebradas em todos os cenários.
- O ataque de dicionário permitiu quebrar passwords adicionais (trudy e alice).
- A password da mallory só foi quebrada quando se utilizou uma wordlist contendo diretamente as cinco possíveis passwords.

2

Exercício 2

2.1 Estado de autenticação no Moodle

O objetivo deste exercício é analisar de que forma o Moodle mantém o estado de autenticação após o utilizador iniciar sessão no browser. A análise é feita com base na informação visível nas ferramentas de desenvolvimento do browser.

Mecanismo de manutenção de sessão

A aplicação mantém o estado de autenticação recorrendo a um **cookie de sessão** denominado `MoodleSession`. Este cookie contém um identificador anónimo gerado pelo servidor no momento do login. O cookie tem as seguintes características:

- É enviado automaticamente pelo browser em todos os pedidos HTTP ao domínio do Moodle;
- Contém apenas um identificador de sessão, não inclui informação sensível do utilizador;
- Permite ao servidor associar cada pedido ao contexto de sessão autenticada, recuperando a informação correspondente armazenada no backend;
- Utiliza flags de segurança como `HttpOnly` e `Secure`, impedindo acesso via JavaScript e garantindo envio apenas por HTTPS.

Deste modo, o servidor mantém toda a informação da sessão do lado backend, enquanto o cookie funciona apenas como uma referência que permite mapear cada pedido à sessão do utilizador.

2.2 Acesso ao perfil usando outro cliente

Acesso via Postman ou curl

Um cliente externo, como o Postman ou o `curl`, consegue aceder à área autenticada do utilizador desde que envie o mesmo cookie de sessão `MoodleSession` que o browser utiliza.

O acesso pode ser realizado enviando o cookie no cabeçalho HTTP `Cookie`. Por exemplo, usando o `curl`:

```
 curl -H "Cookie: MoodleSession=<valor_do_cookie>"
https://2526moodle.isel.pt/my/
```

Como o estado da sessão se encontra armazenado no servidor, e o cookie apenas transporta o seu identificador, o Moodle reconhece o pedido como pertencente ao mesmo utilizador autenticado no browser. Assim, o acesso ao perfil e disciplinas em curso é permitido sem necessidade de repetição do login.

3

Exercício 3

3.1 Integração OAuth2 e OpenID Connect do Calendly

O Calendly permite autenticação e acesso a calendários do utilizador através dos serviços OAuth2 e OpenID Connect da Microsoft e da Google. Para analisar este processo, foram inspecionados os pedidos HTTP efetuados pelo browser durante o fluxo de autorização, recorrendo ao separador *Network* das Developer Tools e ativando a opção *Preserve log*.

3.2 Autenticação com Microsoft

Primeiro pedido ao authorization endpoint

O primeiro pedido feito pelo browser ao servidor de autorização foi identificado observando, nos logs, o primeiro redirecionamento que sai do domínio Calendly e entra num domínio Microsoft. Este pedido aparece tipicamente com método GET e contém os parâmetros do Authorization Code Flow, tais como `client_id`, `redirect_uri`, `scope`, `response_type=code` e `state`.

A partir deste pedido foi obtida a seguinte informação:

- **URI do authorization endpoint**: Observada diretamente no pedido GET inicial: https://login.microsoftonline.com/common/oauth2/v2.0/authorize
- **client_id do Calendly**: Lido no parâmetro `client_id` presente na query string do pedido: `751ff9b5-edde-4dc1-8093-adf647495745`
- **Scopes solicitados**: Identificados no parâmetro `scope` do mesmo pedido. O browser apresenta este parâmetro com os valores codificados em URL; após decodificação, obtiveram-se: `openid email profile offline_access https://graph.microsoft.com/User.Read`

`https://graph.microsoft.com/Calendars.ReadWrite`
`https://graph.microsoft.com/Calendars.ReadWrite.Shared`

- **Parâmetro state**: O parâmetro `state` surge no mesmo pedido GET e serve para proteção contra CSRF. O valor observado foi: `908b7026e06ea282e0b467c38b369913844113f56e08640b`

Último pedido ao servidor de autorização

O último pedido ao servidor Microsoft corresponde ao redirecionamento final após o utilizador introduzir credenciais e autorizar o Calendly. Este pedido foi identificado como o último pedido cujo domínio ainda pertence ao servidor de autorização Microsoft antes do redirecionamento para o domínio do Calendly. Não contém o `code`; apenas confirma a última etapa de autenticação antes do envio para o `redirect_uri`.

Pedido para a callback do Calendly

O pedido feito pelo browser para o endpoint de callback surge imediatamente após o último pedido ao domínio Microsoft. Este pedido foi identificado como um GET para um domínio Calendly com a presença dos parâmetros `code` e `state`. A partir desse pedido obteve-se:

- **URI de callback**: Observada diretamente como o URL de destino do pedido: https://calendly.com/users/auth/azure2_oauth2/callback
- **Parâmetro code**: Lido no parâmetro `code` presente apenas neste pedido de callback: `M.C555_BAY.2.U.02d23d5a-9210-465e-1540-1671d973b716`

3.3 Autenticação com Google

O processo foi repetido seguindo exatamente o mesmo método: inspecionar o primeiro pedido ao authorization endpoint, o último pedido ao servidor Google e o pedido para o callback do Calendly.

- **URI do authorization endpoint**: Obtida diretamente do primeiro pedido GET para o domínio Google: https://accounts.google.com/v3/signin/accountchooser
- **client_id**: Lido no parâmetro `client_id` da query: `797340822162.apps.googleusercontent.com`
- **Scopes**: Nos pedidos iniciais, os scopes estão na query do authorization request e incluem: `email profile https://www.googleapis.com/auth/calendar` No pedido de callback, verificam-se scopes expandidos incluindo `openid`, `userinfo.profile` e `userinfo.email`, visíveis no payload refletido no redirecionamento.
- **URI de callback**: Observada diretamente no pedido de redirecionamento final: https://calendly.com/users/auth/google_oauth2/callback
- **Parâmetro state**: Identificado na query do primeiro pedido: `e1da99902960d889f323318cd5ea78495eadebb8921f6039`
- **Parâmetro code**: Observado apenas no pedido para o callback: `4/0Ab32j92IlwBZmRj1oxRANdk6GSDYtazH-59Fuk0je7ePmv03H2LSbXUBy-WmfYutmESRsw`

3.4 Comparação entre Microsoft e Google

Client_id

Os valores não coincidem, pois cada provedor gere o seu próprio registo de aplicações e atribui um `client_id` independente, mesmo tratando-se da mesma aplicação (Calendly).

Scopes

Os scopes diferem porque cada provedor oferece APIs diferentes. A Microsoft utiliza scopes do Microsoft Graph; a Google utiliza scopes das Google APIs. Ambos incluem permissões de perfil e calendário, mas não são semanticamente equivalentes.

URI de callback

O Calendly define uma `redirect_uri` distinta para cada provedor, permitindo isolamento entre fluxos de autenticação e configurações próprias em cada integração.

3.5 Presença do access token e id token no browser

Não é possível encontrar o *access token* ou o *id token* nos pedidos registados pelo browser.

O fluxo utilizado é o Authorization Code Flow. No browser só circulam o `code` e o `state`. A troca do `code` por tokens é realizada pelo backend do Calendly através de um pedido direto ao token endpoint dos provedores. Como consequência, os tokens nunca são expostos ao browser, aumentando a segurança do processo.

4

Exercício 4

4.1 Política de controlo de acessos

Considere-se a política de controlo de acessos definida sobre o modelo RBAC1:

- Conjunto de utilizadores: $U = \{u_1, u_2, u_3\}$
- Conjunto de roles: $R = \{r_0, r_1, r_2, r_3, r_4\}$
- Conjunto de permissões: $P = \{p_a, p_b, p_c, p_d, p_e\}$
- Hierarquia de roles: $\{r_0 \preceq r_2, r_1 \preceq r_3, r_2 \preceq r_4, r_3 \preceq r_4\} \subseteq RH$
- Associação utilizador-role: $UA = \{(u_1, r_1), (u_2, r_2), (u_3, r_4)\}$
- Associação role-permissão: $PA = \{(r_0, p_a), (r_1, p_b), (r_2, p_c), (r_3, p_d), (r_4, p_e)\}$

Seja s_2 um identificador de sessão tal que $user(s_2) = u_2$, onde foram ativadas todas as roles associadas ao utilizador u_2.

4.2 Análise de acesso

O utilizador u_2 está associado à role r_2. Assim, as roles ativas em s_2 são:

$$Roles(s_2) = \{r_2\}.$$

A role r_2 possui as seguintes permissões:

- Diretamente via PA: p_c

- Herança de roles inferiores na hierarquia: $r_0 \preceq r_2 \implies p_a$

Portanto, o conjunto de permissões disponíveis em s_2 é:

$$Perm(s_2) = \{p_a, p_c\}.$$

4.3 Conclusão

Não é possível usar s_2 para aceder ao recurso X que exige a permissão p_e, porque $p_e \notin Perm(s_2)$.

Todas as permissões **não disponíveis** em s_2 são:

$$\{p_b, p_d, p_e\}.$$

Isto decorre do facto de que r_2 não herda permissões de r_3 nem de r_4, pelo que não dispõe das permissões p_b, p_d, p_e.

5

Exercício 5

5.1 Configuração de um servidor HTTPS e autenticação de cliente

O objetivo deste exercício consiste em configurar um servidor HTTPS recorrendo ao código disponibilizado no repositório da disciplina, utilizando o certificado e chave privada correspondentes ao domínio `www.secure-server.edu`, emitidos pela CA1-int do primeiro trabalho. O servidor é configurado em dois cenários distintos: 1) sem autenticação de cliente 2) com autenticação de cliente usando o certificado `Alice_2`.

A implementação do servidor, realizada em Node.js, permite ajustar diretamente os parâmetros do handshake TLS através do objeto `options`. A configuração inclui a chave privada do servidor, o respetivo certificado e a cadeia da autoridade certificadora responsável pela sua emissão.

5.1.1 Configuração sem autenticação de cliente

No primeiro cenário, o servidor HTTPS é configurado para não exigir qualquer certificado ao cliente. Neste modo, os parâmetros relevantes do handshake são:

- `requestCert = false`: o servidor não solicita certificado ao cliente.
- `rejectUnauthorized = false`: mesmo que algum certificado fosse enviado, não seria verificado.

Nesta configuração, a comunicação estabelece-se garantindo confidencialidade e integridade, mas sem autenticação do cliente.

A ligação foi testada com um browser, confirmando que o acesso ao servidor é realizado sem qualquer pedido de certificado. O ecrã correspondente encontra-se aqui:

Para esta configuração, adicionamos o certifcado CA1 à lista de cerificados de autoridades de certificação de raiz fidedigna do sistema operativo.

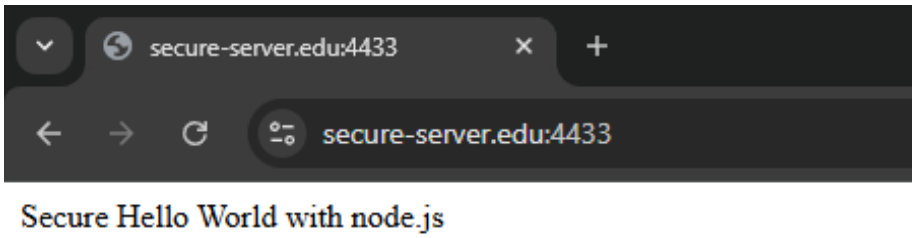

Figura 5.1: Ligação bem-sucedida ao servidor HTTPS sem pedido de certificado do cliente

5.1.2 Configuração com autenticação de cliente

No segundo cenário, o servidor é configurado para exigir autenticação mútua TLS (mTLS). Os parâmetros alterados foram:

- `requestCert = true`: o servidor solicita um certificado ao cliente.
- `rejectUnauthorized = true`: o servidor valida o certificado usando a CA fornecida.
- `ca = fs.readFileSync('ca2.pem')`: é usada a CA que emitou o certificado `Alice_2`.

Com esta configuração, o browser solicita ao utilizador a seleção de um certificado válido, sendo usado o certificado `Alice_2`. A ligação só é concluída se o certificado pertencer à cadeia confiada pelo servidor.

A captura de ecrã demonstrando a ligação com autenticação de cliente encontra-se aqui:

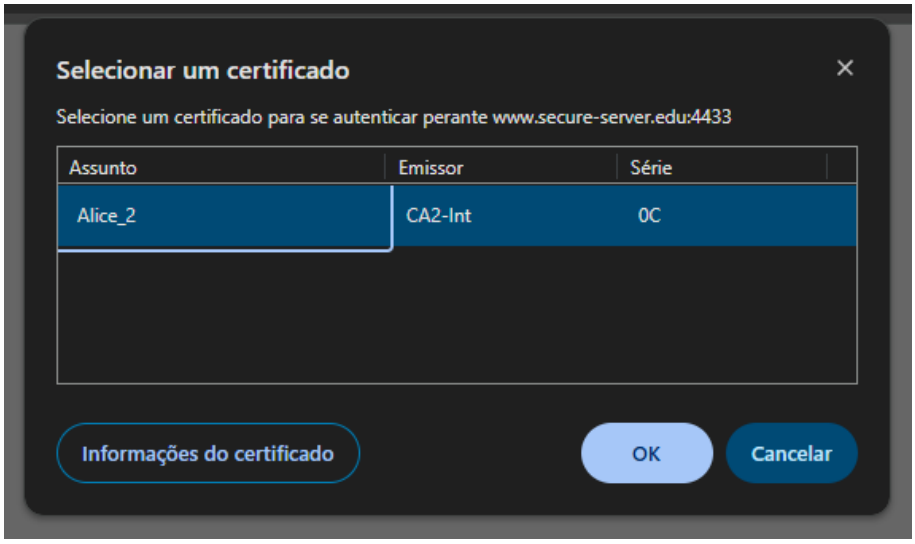

Figura 5.2: Prompt do browser solicitando o certificado do cliente (`Alice_2`)

Após a seleção correta do certificado, a ligação é estabelecida com sucesso.

Para esta configuração, adicionamos a chave privada `Alice_2` à lista de cerificados pessoais do utilizador autenticado no sistema operativo. E, ainda, adicionamos o cerificado CA2-Int à lista de cerificados de autoridades de certificação intermédias do sistema operativo.

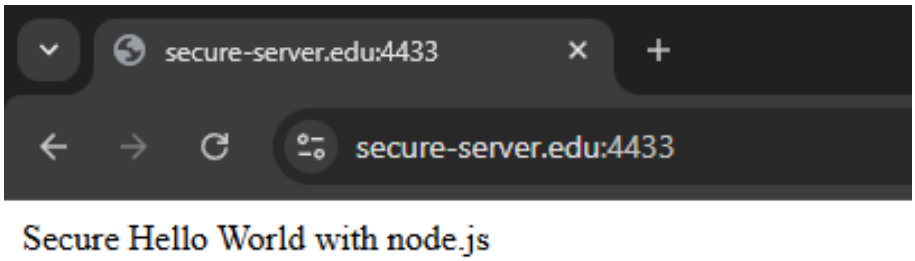

Figura 5.3: Ligação bem-sucedida ao servidor HTTPS com autenticação do cliente

5.1.3 (i) Algoritmos de segurança usados no Record Protocol

Ao ligar o browser ao servidor HTTPS, a análise do Security Overview revela os seguintes parâmetros de segurança:

- **Protocolo**: TLS 1.3
- **Troca de chaves**: X25519 (ECDHE)
- **Cifra de sessão**: AES-256-GCM

Confidencialidade das mensagens: A confidencialidade é assegurada pelo algoritmo AES-256 em modo GCM, um esquema AEAD (Authenticated Encryption with Associated Data) que cifra os dados e impede que um atacante obtenha o conteúdo sem possuir as chaves de sessão derivadas durante o handshake.

Autenticidade e integridade: O modo GCM fornece tanto integridade como autenticidade através do *authentication tag*, detectando qualquer alteração nos dados transmitidos. Em TLS 1.3 não é necessário HMAC separado, pois toda a proteção de integridade é gerida pelo AEAD.

Estabelecimento de chaves e Perfect Forward Secrecy: A troca de chaves é feita via X25519, variante de ECDHE, garantindo Perfect Forward Secrecy (PFS). Mesmo que a chave privada do servidor seja comprometida no futuro, comunicações passadas não podem ser decifradas.

Autenticação dos participantes

- **Sem autenticação de cliente**: apenas o servidor é autenticado através do seu certificado X.509.
- **Com autenticação de cliente**: o browser apresenta o certificado Alice_2, que é validado pelo servidor, garantindo autenticação do cliente.

5.1.4 (ii) Vulnerabilidade da cipher suite TLS_RSA_WITH_AES_128_CBC_SHA256

Esta suite utiliza RSA clássico para troca de chaves. O problema central consiste no facto de:

$$\text{premaster secret} \longrightarrow \text{RSA-encryptado diretamente com } PK_{Server}$$

Se o atacante registar o tráfego TLS e mais tarde obtiver a **chave privada RSA do servidor**, pode executar:

$$\text{premaster secret} = D_{SK_{Server}}(\text{ClientKeyExchange})$$

Depois de recuperar o premaster secret, todo o tráfego TLS pode ser decifrado retroativamente, sendo possível reconstruir as chaves simétricas derivadas na fase de handshake. Este ataque denomina-se *RSA key exchange compromise* e é a principal razão pela qual RSA deixou de ser recomendado para troca de chaves em TLS 1.3, sendo substituído por esquemas ECDHE que garantem confidencialidade perfeita (PFS).

Assim, um atacante com acesso posterior à chave privada do servidor pode decifrar comunicações passadas entre browser e servidor, comprometendo confidencialidade histórica.

5.2 Ligação a servidor HTTPS usando JCA

A alínea 5.2 exige a implementação de um cliente HTTPS em Java utilizando a API JCA (Java Cryptography Architecture), capaz de se ligar ao servidor HTTPS configurado previamente, no modo sem autenticação de cliente.

A aplicação Java carrega uma truststore contendo exclusivamente a CA que emitiu o certificado do servidor. Esta truststore, `roots.jks`, foi obtida no trabalho anterior e substitui a necessidade de criar um novo ficheiro com `keytool`. O cliente realiza:

1. carregamento explícito da truststore;
2. criação de um `TrustManagerFactory`;
3. inicialização de um `SSLContext` com os gestores de confiança;
4. estabelecimento de um `SSLSocket` para o servidor `localhost:4433`;
5. execução do handshake TLS;
6. obtenção da sessão TLS negociada.

Depois da ligação, o programa imprime o protocolo TLS e a cipher suite efetivamente negociada.

A execução da aplicação produziu o seguinte resultado:

```
Protocolo:    TLSv1.3
Cipher Suite: TLS_AES_256_GCM_SHA384
```

A suite `TLS_AES_256_GCM_SHA384` é característica de TLS 1.3 e utiliza AES no modo GCM, que garante simultaneamente confidencialidade, integridade e autenticidade das mensagens sem necessidade de HMAC separado.

6

Exercício 6

6.1 Aplicação Web GitTasks

O exercício 6 consiste na implementação de uma aplicação web denominada **GitTasks**, responsável por integrar dados provenientes de repositórios GitHub com a criação de tarefas no serviço Google Tasks. A aplicação combina autenticação OpenID Connect, controlo de acesso baseado em papéis e interação com APIs externas através de pedidos HTTP.

A solução foi implementada integralmente sem SDKs oficiais, recorrendo apenas a chamadas HTTP diretas (via `axios`), sessões armazenadas em memória e controlo de permissões com Casbin.

6.2 Configuração e variáveis de ambiente

Para executar a aplicação, é necessário criar um ficheiro `.env` contendo as variáveis utilizadas pelo servidor:

```
PORT=3001
SESSION_SECRET=chave_ultra_hiper_mega_secreta_123
HMAC_SECRET=outra_chave_ultra_hiper_mega_secreta_mas_com_hmac_456

GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
OAUTH2_REDIRECT_URI=http://localhost:3001/auth/callback

GITHUB_CLIENT_ID=
GITHUB_CLIENT_SECRET=
GITHUB_REDIRECT_URI=http://localhost:3001/github/callback
```

As variáveis `GOOGLE_CLIENT_ID` e `GOOGLE_CLIENT_SECRET` são obtidas ao criar um cliente OAuth 2.0 na Google Cloud Console. As variáveis `GITHUB_CLIENT_ID` e `GITHUB_CLIENT_SECRET` são geradas ao criar uma OAuth Application em `github.com/settings/developers`. As chaves de sessão (`SESSION_SECRET` e `HMAC_SECRET`) são utilizadas pelo servidor para assinar cookies e proteger conteúdos sensíveis armazenados na sessão.

6.3 Autenticação de utilizadores

A autenticação principal é realizada via Google, utilizando o protocolo *OpenID Connect*. O fluxo seguido pela aplicação é o seguinte:

- O utilizador inicia a autenticação selecionando o botão Google.
- É gerado um valor *state* protegido com HMAC e enviado para o endpoint de autorização do Google.
- Após o utilizador autenticar com a Google, o servidor recebe um código de autorização.
- O código é trocado por *access token*, *refresh token* e *id token*.
- A aplicação pede os dados do utilizador ao endpoint `userinfo`.
- A sessão do utilizador é criada e persistida em memória, incluindo email, nome, avatar, tokens Google e o respetivo papel.

A sessão é mantida com cookies HTTP-only, garantindo isolamento entre utilizadores.

6.4 Autorização e papéis (RBAC)

A aplicação utiliza controlo de acessos baseado em papéis através da biblioteca **Casbin**. Toda a informação de papéis e permissões reside no ficheiro de política (`policy.csv`) e não no código.

6.4.1 Atribuição de papéis

Após o login do utilizador, a aplicação executa:

```
const role = await enforcer.getRolesForUser(email)
```

O papel é obtido diretamente da política Casbin e é armazenado na sessão. Caso o email não exista na política, a autenticação é recusada.

6.4.2 Permissões

As permissões são definidas exclusivamente na política Casbin. Em termos gerais:

- **free**: pode consultar milestones no GitHub.
- **regular**: pode criar tasks numa lista predefinida no Google Tasks.
- **premium**: pode criar tasks em listas cujo nome é definido pelo utilizador.

O middleware `requirePermissions` consulta o enforcer para validar se o utilizador tem permissão para a operação pretendida.

6.5 Integração com GitHub

A aplicação comunica diretamente com a API do GitHub, utilizando pedidos HTTP autenticados quando necessário:

- Listar os repositórios do utilizador.
- Consultar detalhes de um repositório.
- Obter milestones de repositórios públicos ou privados.

Não é utilizado qualquer SDK; todas as chamadas são construídas manualmente com `axios`.

6.6 Integração com Google Tasks

A criação de tarefas no Google Tasks segue as permissões definidas no RBAC:

- O utilizador **regular** cria tasks numa lista pré-definida pela aplicação.
- O utilizador **premium** pode indicar o nome da lista onde pretende criar a task.

O servidor gere automaticamente a expiração dos tokens, recorrendo ao *refresh token* para obter novos access tokens quando necessário.

6.7 Sessões e armazenamento em memória

Toda a informação relativa ao utilizador permanece em memória:

- Dados Google (email, nome, avatar).
- Tokens Google e respetivas datas de expiração.
- Estado da autenticação GitHub.
- Papel do utilizador atribuído via Casbin.

As sessões são identificadas por cookies HTTP-only, impedindo acesso via JavaScript.

6.8 Interface Web

A interface inclui:

- Botões para login Google e GitHub.
- Informação do utilizador autenticado.
- Indicação clara do papel ativo.
- Formulários para criação de tarefas e consulta de repositórios e milestones.
- Comportamento dinâmico controlado pelo papel do utilizador.

6.9 Segurança e controlo de acesso

- Endpoints sensíveis usam o middleware `requireAuth`.
- As permissões são verificadas com `enforcer.enforce(user, object, action)`.
- Tentativas de acesso indevido devolvem códigos HTTP 401 ou 403.

6.10 Resumo

A aplicação GitTasks cumpre todos os requisitos do exercício:

- Integra GitHub e Google Tasks de forma transparente.
- Implementa autenticação completa via OpenID Connect.
- Aplica controlo de acessos RBAC baseado em Casbin.
- Mantém sessões e estados em memória sem necessidade de base de dados.
- Usa apenas HTTP requests diretos, sem dependência de SDKs externos.
- Adapta automaticamente a interface às permissões do utilizador.

A solução final apresenta uma arquitetura robusta, segura, e alinhada com as práticas solicitadas no enunciado.